

TASK – 02

DOCUMENTATION

LEVEL-1:

In Level-1, I implemented a fully functional, interactive OS shell in C. It handles memory allocation, String Tokenization, and process management.

Initially, an infinite loop is created which asks the user to enter the command till the user presses **ctrl+d** to end the loop. Once the input is obtained, then it is passed through string tokenizer. For this, in-built function in C, **strtok** is used. The delimiters are space, new line character and new tab character. The output of each token is then stored in an array.

Once the tokens are stored in the array till the end, then the next part of the shell, implementing the commands entered by the user is done. For this 3 functions **fork()**, **execvp()** and **wait()** are used.

The **fork()** function is used in order to clone the shell itself. This is because the shell can't run programs without dying and the OS can't create new files on its own. So using the **fork()** function, the shell is cloned. Now there will be 2 shells, parent shell and child shell. The **execvp** function is a self-destructive function. So it can't be directly used in the parent shell. So the **fork()** function is used to clone the shell. The child shell carries out the command entered by the user, using **execvp()** function. In the process it destroys itself. During this process, the parent shell remains silent and sleeps. This is done using the **wait()** function.

In order to carry out the built-in commands (cd, exit....), cloning the shell isn't useful. This is because the child shell brings the change, and as it gets destroyed, it takes the changes along with it. As a result, the parent shell remains unchanged. Therefore for built-in commands, the programme must be intercepted before the **fork()** function is called.

For the interception. The built-in function **strcmp** is used. This compares two strings and returns 0 if both of them are equal, a negative number if the first string is less than the second and a positive number if the first string is greater than the second string. Here **args[0]** is compared with the string "cd" and if it is equal, then using **chdir** the current working directory is changed into the file which is mentioned in **args[1]**. If this is successfully executed means, a value of zero will be returned. If a non - zero number is returned as an output means there is an error.

Similarly **args[0]** is compared with the string “exit”, if it is equal means, the programme is exited smoothly.

After implementing the level-1, I cross-verified the code using fuzzing, and after executing for 12 minutes and doing 37,000 executions, there were 0 crashes and 2 hangs. When I analyzed the 2 hangs, it was known to be a false-positive. The fuzzer expects a programme to enter and exit the code at some point, it hung because the fuzzer fed 1000s of commands to the shell and asked the OS to fork it which leads to exhaustion. As a result it is saved as a hang.

LEVEL - 2

After successfully building the shell, the level-2 demanded a way to send texts and files without the notice of the terrorists. So in this level I built a mechanism which handles dual way transmission (sender and receiver) of files and texts.

First I created a struct for holding the description of the data which is being sent. This header data is stored in the **file_header** struct. The first 4 bytes **ver_bytes** stores the value “NIT\x00”. The next 64 bytes store the file data. The final 4 bytes store the information about the file size. As a result the entire size of the header is 72 bytes.

By default, the C compiler adds invisible padding bytes to structs to optimize CPU memory reads. If GCC padded my struct to 76 bytes, it would misalign the network stream and corrupt the file transmission. To prevent this, I applied the **__attribute__((packed))** compiler directive in my header file, strictly locking the struct to exactly 72 bytes.

If we send the file size in **uint32_t** is sent as a raw memory means, different machines with different architectures will read it different sizes. A small megabyte file might be read as gigabytes. So, in order to avoid that, we use **htonl()** function, which converts the number from host system type to a format which is suitable in sending across the network. Upon receiving the header data, the receiver might use the **ntohl()** function, which converts the network formatted file size to **long** type.

When the sender initially sends the file, the socket number i.e. **client_id** is obtained using the **socket()** function actually takes the domain (AF_INET), the type (SOCK_STREAM), and the protocol (0). All these 3 variables are stored in a **struct** named **target_addr** which is of the in-built formatted struct **sockaddr_in** in LINUX. The target IP address is then converted from a readable string to a computer readable binary form using **inet_pton**. Then the file is opened using the file name provided by the user in “**read binary**” mode. The size of the file is found out by setting the pointer to the end of the file and setting the position into a **uint32_t** variable by using **ftell()** function. A new **struct send_file** is formed of the format **file_header**. This is then stored after converting it to the suitable format which can be sent across the network in the file size variable of the **send_file** struct. The initial “**NIT\X00**” and the filename is also stored in the **send_file struct**. This struct is then sent to the receiver using the send function.

After that, a temporary array is created of size 4096 bytes. Then a while loop is set. Data from the source file is then scooped into the temp array till it gets completed using **fread()** function. The output of the **fread()** is stored in a **bytes_read** variable. Then it is sent to the receiver.

From the receiver's side, the **server_id** is obtained using the **socket()** function which takes the input as **family of addresses(IPv4)**, **SOCK_STREAM**. Then a struct **server_addr** is created which is of the format **sockaddr_in**. Then the **bind()** function is called. This first takes the **server_id** plugs in with the **IP address and the port no.** It uses type cast because it is an universal function and it should accept all family of addresses. Then the listen function is called. This turns the socket on and wait for the incoming connections, with maximum 5 connections at a time. Then a variable **client_socket** is made to hold the FILE DESCRIPTOR number which tells the receiver to which connection to listen to. After that, the header file is received using **recv()** function. If the first 4 bytes are “**NIT\x00**”, then the entire file is received using **the recv()** function. The received data is then entered into a new file which is opened in “**write binary**” mode.

LEVEL - 3

In level-3 I developed a cryptographic method which encrypts the file the sender sends, in order to prevent the explicit availability of the data transferred to the terrorists. For this I followed the **Diffie-Hellman** method of encryption. In this method, initially two public parameters, a large prime and the base integers are created as a constant in the header file.

We are randomly generating a 64 bit number by opening a file using **fopen()** function with the file address being **"/dev/urandom"** in **"read binary" mode**. The value inside the file is then transferred into a variable **"priv1"**. Finally, the private key is obtained by performing modular division with the prime number and 1 is added to the end result. This is because, as per **Fermat's Little theorem**, modular exponents always loop back to **prime - 1** exactly. We are adding 1 because to take care if the random private key is exactly divisible by **prime-1**. In this method, private keys are generated on both the sides (sender and the receiver).

Using this the public key is calculated by both the sender and receiver. The public keys which are calculated are sent to each other using **send() function** and received using **recv() function**.

The public keys are calculated by the

$$A=[(\text{base})^{(\text{private_key})}] \% \text{prime_number}$$

The public key A is sent to the other person.

In order to compute the modular arithmetic problem, we are using the **SQUARE AND MULTIPLY ALGORITHM**. This is because we cannot directly use the in-built **pow()** function, as computing the power of a 64 bit number is a hectic process and in doing so the computer will return a garbage data.

In the **SQUARE AND MULTIPLY ALGORITHM**, we are computing the modular division in the bitwise level. This makes the computing easier. In this method we declare a variable **result** and **set it to 1**. Then a **while loop** is set to implement the square and multiply algorithm. Then, the exponent and 1 is made to undergo bitwise AND operator to check if the bit of the exponent is 1. The result is multiplied with the base and performed modulus division with the prime number. Then comes the squaring step. This is the squaring step in the square and multiplying algorithm. There is a type cast on base because when squaring

takes place, it might be more than 64 bytes causing overflow. Then the exponent is made to do a right shift by 1. This process is continued till the exponent is greater than 0. At the end, **result** is the required answer.

Once both the public keys are shared, the common key is computed using the same modular arithmetic formula : **[(PUBLIC_KEY SENT BY THE OTHER PERSON)^(YOUR PRIVATE KEY)] % prime_number.**

Then the public key is shifted using a linear congruential generator. This is done in order to ensure that For every byte of the file processed, the generator must output exactly one byte of pseudorandom data. This is done by creating a multiplier variable and an incrementer variable. Both of them are set to a constant value. The common key is updated by multiplying the multiplier and added by the increment, resulting in a 64 bit output. The output is 64 bit, but we need only 8 bits, so we are shifting by 56. Therefore the keystream is generated.

Once this done, the encryption path becomes easier. We take a file byte and it is **XORed** with the corresponding keystream byte. This ensures that the data is encrypted. For decryption we again **XOR** the cipher text with the same corresponding keystream byte. This will give us the plain text. This method works because we know that **XOR** is commutative.

LEVEL - 4

In level-4 I built a passive intrusion detection system that alerts whenever an external unauthorized machine probes into our laptop. In this tripwire system, I used **RAW SOCKET** instead of normal socket. This allows the receiver to directly intercept the user data even before the **OS** operates on it. The size of data is stored in the variable **data_size** which is obtained via **recvfrom()** function.

If the value of the variable **data_size** is less than 20 (which is the size of the struct **iphdr**) means, the TCP packet is ignored. We are checking with the size 20 because the minimum size of the IP HEADER is 20 and if it is lesser than it means, the packet is ignored.

If it is greater than 20 means, the IP address is stored. Then the IP address is converted into a readable string from computer readable bytes. Then the IP address is compared against the hardcoded TEAMMATE's IP address. If it is not equal means error is printed with red colour, indicating that there is an intrusion and we need to close the program to prevent getting caught.